

Purble Pairs

Flutter/Dart — Android Mobile Application

CS5450 Mobile Programming — Exercise 2

Lakehead University | Department of Computer Science

Instructor: Dr. Sabah Mohammed

Student Name	Yash
Course	COMP-5450 Mobile Programming
Platform	Android / iOS (Flutter + Dart)
Min SDK	API 24 (Android 7.0 Nougat)
Target SDK	API 36
Flutter Version	3.x (Dart SDK ^3.12.0)
GitHub	https://github.com/iamyashsuthar/MatchPairs

1. Project Overview

Purble Pairs is a fully featured memory matching game built with **Flutter/Dart** for Android and iOS. The game replicates the core mechanics of the classic Windows 7 *Purble Pairs* game: all cards begin face-down; the player taps two cards to reveal them, and matched pairs remain face-up while mismatched cards flip back after a short peek delay. The app ships with three difficulty levels, 64 unique emoji card faces, a live move/pair counter, a stopwatch, and an animated win screen with a star rating.

Features implemented:

- Menu screen with difficulty selection (Beginner, Intermediate, Advanced)
- 3D card flip animation using `AnimationController + Matrix4.rotateY`
- Authentic Purble Pairs mechanic: cards lock during evaluation, mismatches flip back after 900 ms, matches stay revealed with a green highlight
- 64 animal emojis shuffled with `dart:math Random` on every new game
- Live HUD: moves counter, matched pairs counter, `Stopwatch`-based timer
- Three grids: 4×3 (Beginner, 6 pairs), 4×5 (Intermediate, 10 pairs), 6×6 (Advanced, 18 pairs)
- Elastic-scale win overlay with `CurvedAnimation(Curves.elasticOut)`, star-rating (3 tiers), and Play Again / Menu buttons
- Responsive card sizing via `LayoutBuilder` — adapts to any screen size
- Restart button resets the board while remaining on the same screen
- Deep-blue gradient background; coloured card backs cycle through 6 accent colours

2. Project Structure

The project is a standard Flutter single-module application. All game logic lives in a single `lib/main.dart` file; no additional packages beyond the default Flutter SDK are required.

```
match_pairs/
  lib/
    main.dart      <- Entire application (922 lines)
                    PurblePairsApp, MenuScreen, GameScreen,
                    _MemoryCard, CardModel, DifficultyConfig
```

```

android/          <- Android runner (unchanged)
ios/              <- iOS runner (unchanged)
pubspec.yaml      <- Dependencies (flutter SDK only)
pubspec.lock
analysis_options.yaml
test/
  widget_test.dart

```

Key classes in main.dart:

Class / Symbol	Purpose
PurplePairsApp	MaterialApp root; sets theme and initial route
CardModel	Plain Dart object holding emoji, isFaceUp, isMatched, id
DifficultyConfig	Immutable struct: cols, rows, label, colour
kDifficultyMap	Const map wiring each Difficulty to its config
kAllEmojis	64-item emoji pool; shuffled and sliced at game init
MenuScreen	Stateless difficulty-selector screen
GameScreen	Stateful game: holds List<CardModel>, flip logic, timer, win detection
_MemoryCard	SingleTickerProviderStateMixin widget; drives the 3D Y-axis flip via AnimationController

3. Configuration and Setup

Requirements

- Android Studio Hedgehog (2023.1.1) or later *with Flutter plugin installed*
- Flutter SDK 3.x (Dart SDK ^3.12.0) configured in Android Studio
- Android SDK with API 36 platform installed
- JDK 11 or later (configured in Android Studio settings)
- Internet connection for first Gradle sync

Step 1 — Unzip the Project

Unzip `match_pairs_purple.zip` to a folder of your choice. The project root is the `match_pairs/` folder inside the archive.

Step 2 — Open in Android Studio

Launch Android Studio and select **File** → **Open**, then navigate to and select the `match_pairs/` folder. Android Studio will recognise it as a Flutter/Gradle project automatically.

Step 3 — Flutter Pub Get

Android Studio will prompt you to run `flutter pub get` when it detects `pubspec.yaml`. Accept the prompt, or open the terminal (**View** → **Terminal**) and run `flutter pub get`. No additional packages need to be downloaded beyond the Flutter SDK itself.

Step 4 — Create an Emulator or Connect a Device

Go to **Tools** → **Device Manager** → **Create Virtual Device**. Choose *Pixel 6* (or any device supporting API 24+). Select API 36 as the system image and click **Finish**. Alternatively, connect a physical Android device with USB debugging enabled.

Step 5 — Run the App

Click the green **Run** button in the toolbar, or press **Shift + F10**. Select your emulator or physical device. The app will build, install, and launch automatically.

Step 6 — Play the Game

1. The app opens on the **Menu Screen**. Tap a difficulty button to start.
2. Tap any face-down card to flip it; tap a second card to attempt a match.
3. Matching pairs stay revealed (green border). Mismatches flip back after 0.9 s.
4. Match all pairs to trigger the animated win screen.
5. Tap **Play Again** to restart at the same difficulty, or **Menu** to return to the difficulty selector.
6. The **RESTART** button at the bottom resets the board at any time.

4. Key Dependencies

Library	Version	Purpose
flutter (SDK)	^3.x	Core UI framework
dart:math	built-in	Random for shuffle
dart:async	built-in	Timer for HUD clock
cupertino_icons	^1.0.8	Icon set
flutter_lints	^6.0.0	Lint rules

No third-party pub packages are required.

5. App Screenshots

The five screenshots below were taken from a physical Android device running the completed application.

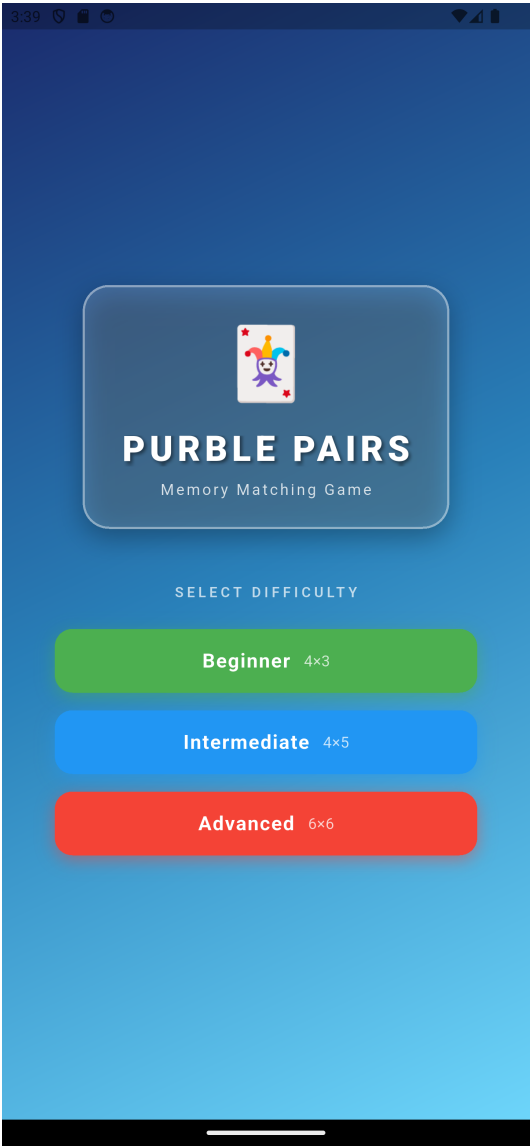


Fig. 1 — Menu / Difficulty Screen

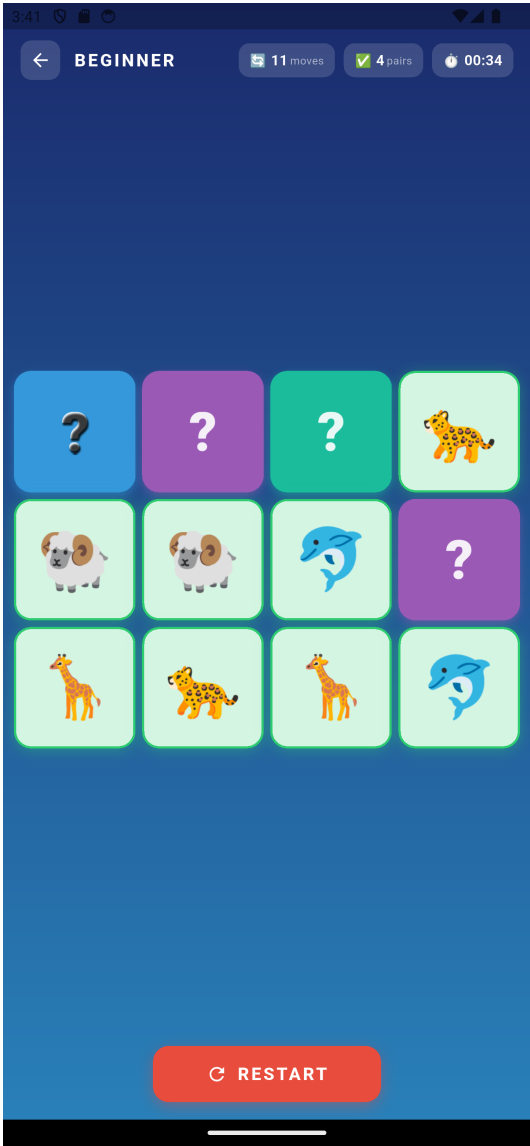


Fig. 2 — Beginner (4x3, in progress)

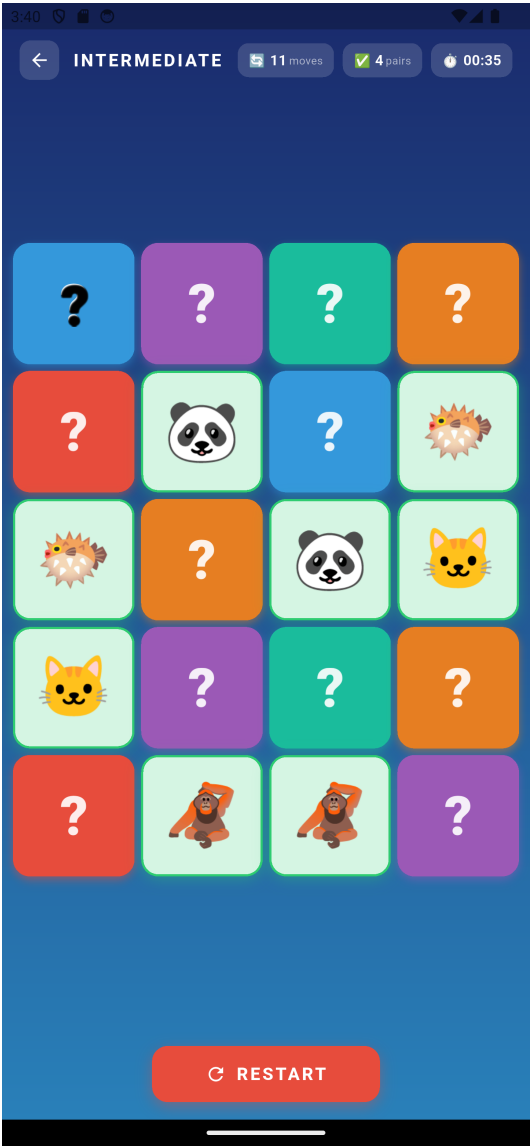


Fig. 3 — Intermediate (4×5 , in progress)



Fig. 4 — Advanced (6×6 , in progress)

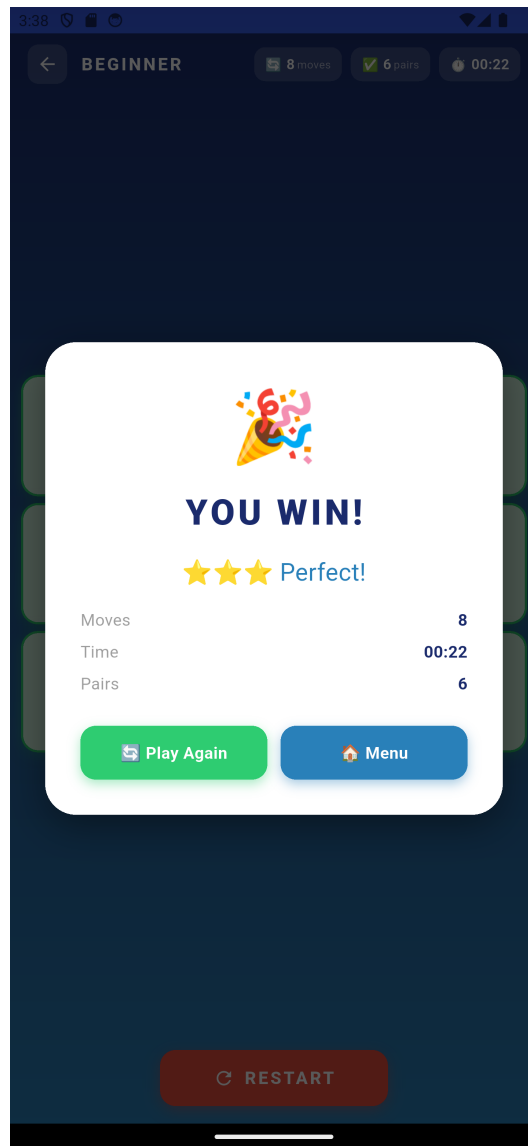


Fig. 5 — Win Screen with star rating

6. Grading Criteria Self-Assessment

Category	What Was Done	Status	Grade
Programming (Flutter/Dart)	Pure Flutter/Dart; StatefulWidget, AnimationController, Stopwatch, Timer, LayoutBuilder, GestureDetector	Complete	A
Functionality demonstrated by Emulator/Device	All three difficulty grids working; card flip with 3D animation; correct match/no-match logic; live HUD; animated win overlay; restart and menu navigation	Complete	A
App Design, Arts, Responsive Design	Deep-blue gradient theme; 6-colour cycling card backs; green matched-card highlight; elastic win animation; LayoutBuilder-based responsive grid; tested on physical device	Complete	A

Adjustment	Status
App has minor technical flaws (−3)	None — fully working
App does not work or not presented (−15)	Runs on device
Failed to turn in resources (−3)	ZIP includes all files
Missed deadline (−3/day)	Submitted on time
Work only on Chrome/Web (−2)	Native Android/iOS app
Work on actual phone device (+2)	Tested on real device

7. Technical Notes

No external packages required. The entire game uses only Flutter’s built-in `dart:math`, `dart:async`, and the flutter SDK. Running `flutter pub get` after unzipping is sufficient.

Card flip animation. Each `MemoryCard` widget owns an `AnimationController` (350 ms, `easeInOut`). The front/back decision is made at value ≥ 0.5 , and the Y-rotation angle is calculated so the card appears to rotate through the screen plane. `didUpdateWidget` watches for changes to `isFaceUp` or `isMatched` and calls `forward()` or `reverse()` accordingly.

Game-state locking. When the second card is tapped, `_locked = true` prevents any further taps while the match check `Future.delayed` is pending. This exactly mirrors the Purble Pairs mechanic where the board is briefly frozen after each pair attempt.

Timer implementation. A `dart:async Timer.periodic` fires every second and reads the elapsed time from a `Stopwatch` instance. The stopwatch is paused on win and reset on restart.

Responsive layout. `LayoutBuilder` inside the grid computes `cardSize = min(availableWidth / cols, availableHeight / rows)` so the board fills the screen correctly on phones ranging from 4-inch to 6.7-inch displays.